

BLOOM: Bit-Slice Framework for DNN Acceleration with Mixed-Precision

Fangxin Liu^{1,2,†,*}, Ning Yang^{1,2,†}, Zongwu Wang^{1,2,*}, Xuanpeng Zhu³, Haidong Yao³, Xiankui Xiong³,
Li Jiang^{1,2}, and Haibing Guan¹

¹Shanghai Jiao Tong University ²Shanghai Qi Zhi Institute

³State Key Laboratory of Mobile Network and Mobile Multimedia Technology, ZTE Corporation, Shenzhen, China

*Corresponding Author {liufangxin, yn937391832, wangzongwu}@sjtu.edu.cn

Abstract—Deep neural networks (DNNs) have revolutionized numerous AI applications, but their vast model sizes and limited hardware resources present significant deployment challenges. Model quantization offers a promising solution to bridge the gap between DNN size and hardware capacity. While INT8 quantization has been widely used, recent research has pushed for even lower precision, such as INT4. However, the presence of outliers—values with unusually large magnitudes—limits the effectiveness of current quantization techniques. Previous compression-based acceleration methods that incorporate outlier-aware encoding introduce complex logic. A critical issue we have identified is that serialization and deserialization dominate the encoding/decoding time in these compression workflows, leading to substantial performance penalties during workflow execution. To address this challenge, we introduce a novel computing approach and a compatible architecture design named “BLOOM”. BLOOM leverages the strengths of the “bit-slicing” method, effectively combining structured mixed-precision and bit-level sparsity with adaptive dataflow techniques. The key insight of BLOOM is that outliers require higher precision, while normal values can be processed at lower precision. By interleaving 4-bit values, we efficiently exploit the inherent sparsity in the high-precision components. As a result, the BLOOM-based accelerator outperforms the existing outlier-aware accelerators by an average $1.2 \sim 4.0\times$ speedup and $24.6\% \sim 71.3\%$ energy reduction, respectively, without model accuracy loss.

I. INTRODUCTION

Deep Neural Networks (DNNs) have made remarkable strides in recent years, delivering exceptional performance across various applications [1]–[5]. However, the increasing complexity and size of DNN models have introduced significant challenges in model inference efficiency. The size of these models has been expanding exponentially, with a staggering 15-fold increase every two years [6]–[8], far outpacing hardware advancements and resulting in escalating computational costs for inference tasks. This widening gap between model size and hardware capabilities necessitates the development of efficient model compression techniques.

Model compression has emerged as a promising method to alleviate the computational and storage burden of DNNs [9].

† These authors contributed equally. This work was supported by the National Key Research and Development Program of China (2024YFE0204300), National Natural Science Foundation of China (Grant No.62402311), Natural Science Foundation of Shanghai (Grant No.24ZR1433700), and ZTE Industry-University-Institute Cooperation Funds (Grant No. IA20240409010).

One of the most popular approaches is quantization, which reduces the data precision to compress models and accelerate computations. While quantization has shown impressive results in Convolutional Neural Networks (CNNs), its effectiveness in attention-based Transformer models has been less pronounced. Recent studies [10], [11] suggest that Transformer models exhibit greater variation in parameter distributions than CNNs and are more sensitive to a small fraction of extreme values. As a result, quantizing these models often requires higher bit widths, such as 8 or 16 bits, compared to the typically lower precision demands of CNNs [12]–[15].

In recent years, various compression techniques have been explored in the design of DNN accelerators. For example, as illustrated in Figure 1(a), GOBO [10] and OLAcel [16] use coordinate lists to track the location of significant values in matrices, while BiScaled-DNNs [17] leverage block sparse index formats to store these indices. OliVe [11] manually constructs significant-sparse value pairs by tightly binding patterns to the parameters, and SPARK [18] embeds indicator bits into parameters to reduce storage while maintaining alignment. However, as shown in Figure 1(b), these methods often introduce complex decoder/encoder designs and significant deserialization/serialization overhead to manage the diverse parameter distributions.

While compression-based encoding techniques effectively separate normal from significant values, they miss opportunities for further energy reduction and accuracy improvement by ignoring sparsity in the higher-order bits of quantized values. For example, an INT8 value can be split into two 4-bit parts, where the higher-order part exhibits greater sparsity than the original value. This observation motivates us to propose a software and architecture solution that efficiently handles the sparsity within the higher-order bits of quantized values.

This paper introduces **BLOOM**, a precision-aware bit-slicing scheme that splits neural network data into specific bit paradigms, achieving mixed-precision representation through an interleaved computing architecture. By exploiting the inherent sparsity within quantized parameters, BLOOM significantly reduces storage and computation demands while maintaining model accuracy, without incurring any serialization or deserialization costs. As illustrated in Figure 1(c), unlike previous compression-based methods, BLOOM does

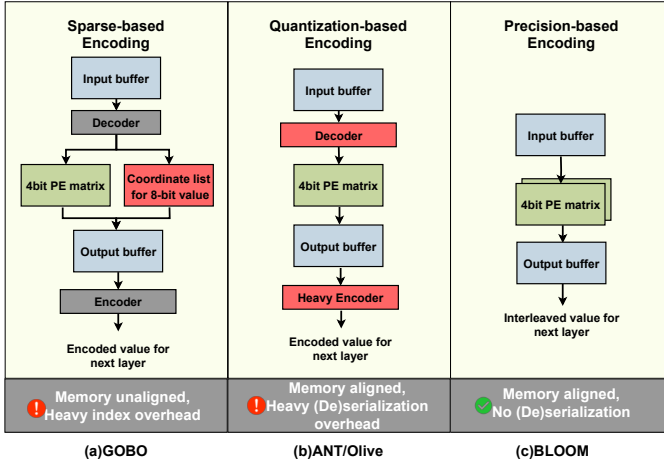


Fig. 1: Compression-based encoding comparison.

not require additional lists or complex encoder/decoder setups. Instead, it ensures efficient storage alignment and computation through interleaved data management.

In summary, we make the following contributions:

- We analyze the current progress in DNN compression research and identify two key challenges. To address these, we propose BLOOM, a precision-aware bit-slicing framework based on the data distribution and bit-level sparsity. BLOOM exploits the inherent high-bit sparsity in models to reduce storage requirements and enhance computational efficiency, all while preserving accuracy.
- We design a hardware-friendly, memory-interleaved computing architecture that organizes PE arrays through precision-aware processing. This ensures both computational accuracy and efficiency, without introducing any serialization or deserialization overhead.
- We implement and evaluate BLOOM, demonstrating through comprehensive experiments that it surpasses existing quantization-based encoding schemes and hardware accelerators in terms of efficiency and overall benefits. Our results highlight BLOOM's effectiveness and scalability in real-world applications.

II. BACKGROUND AND CHALLENGE

A. Outlier of DNN

For DNNs, parameter distributions generally follow a Gaussian pattern, which makes them suitable for quantization [11], [19], [20]. However, a small percentage of outliers—although rare—pose significant challenges for effective quantization. As shown in Figure 2(a), for one layer in ResNet-50, over 99% of parameters are concentrated within three standard deviations (3σ), while outliers extend beyond 15 standard deviations ($> 15\sigma$). This issue is even more pronounced in transformer models like BERT, as illustrated in Figure 2(b).

If these outliers are properly managed, the vast majority of data can benefit from higher computational accuracy or reduced computational overhead. However, outliers often have a disproportionate impact on the overall accuracy of the neural

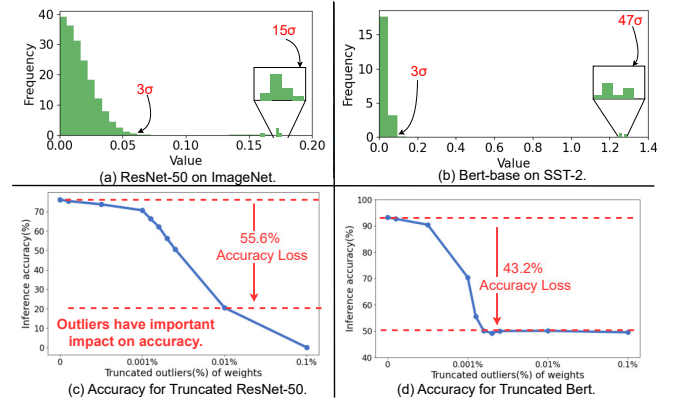


Fig. 2: (a) and (b): Outlier Comparison of CNN model and Transformer model. (c) and (d): Model Accuracy of Truncated ResNet-50 on ImageNet and Bert on SST-2.

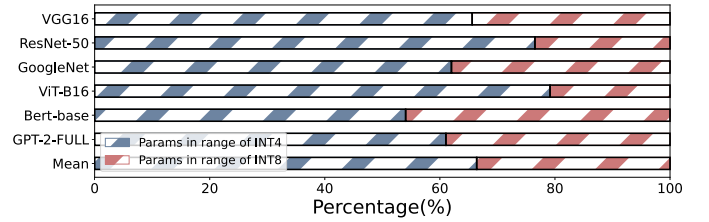


Fig. 3: In INT8 format quantized networks, the percentage of INT4 and INT8 of activation.

network. Simply truncating or removing them can lead to irrecoverable accuracy loss. Figure 2(c) and (d) demonstrate that even directly removing as few as 0.001% of outliers can significantly degrade inference accuracy.

To preserve accuracy, outliers typically require higher-bit quantization, such as 8 or 16 bits [21]. However, this introduces bit-level redundancy for most parameters, which could be represented with lower bitwidths. In Figure 3, we show that after INT8 quantization, most parameters in both CNN-based and attention-based models can be efficiently represented using only 4 bits. Maintaining 8 bits or more for all parameters leads to unnecessary overhead.

Challenge 1. DNNs exhibit inherent bit-level sparsity, but outliers make it difficult to fully exploit this sparsity.

B. Compression-based Encoding

As discussed in the previous section, effectively handling outliers is crucial for the success of quantization [10], [11], [16], [18], [21], [22]. Several compression-based encoding techniques leveraging mixed-precision quantization have been proposed: 1) Isolating outliers based on sparsity encoding: For instance, GOBO [10] uses coordinate lists to track different data representations and applies high-precision quantization to larger values. However, this approach incurs additional overhead for maintaining these lists and managing unaligned memory due to outlier removal. 2) Parameter modifications based on outlier-aware quantization: Olive [11], for example,

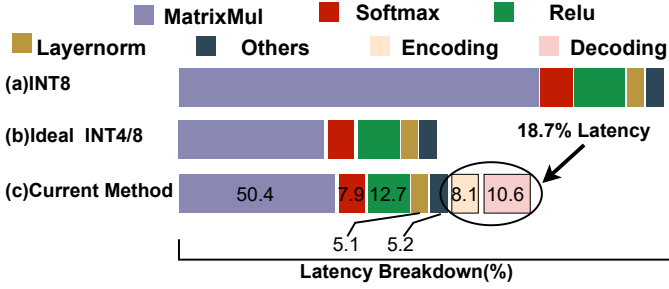


Fig. 4: Latency breakdown for different quantization methods.

applies strict constraints on parameter distribution by pruning adjacent values and storing larger ones, while SPARK [18] embeds indicator bits into parameters and uses a precision compensation mechanism to ensure mixed precision and memory alignment.

Despite reducing computational overhead, these compression techniques increase hardware complexity. The complex encoding and decoding processes introduce additional computational costs, which diminish the potential benefits of hybrid quantization. Figure 4 illustrates the performance penalties across different accelerators, showing that none achieve optimal computational efficiency.

Challenge 2. Encoding and decoding mechanisms incur serialization and deserialization overheads, which offset the performance gains from mixed precision acceleration.

Thus, due to overheads in storage, computation, or serialization/deserialization, a significant gap remains between the theoretical acceleration potential of neural network compression and the actual performance of current accelerator designs. This gap highlights a promising area for future research in DNN compression.

III. ANALYSIS AND MOTIVATION

Quantization is an essential step for deploying neural network models, but current compression-based acceleration faces a significant challenge: **Compression Trap**. Neural networks, especially Transformers, contain non-negligible outliers that require high bitwidths to preserve accuracy, leading to substantial computational and storage redundancy. Existing compression-based methods try to reduce this redundancy by isolating or handling outliers. However, these approaches introduce serialization and deserialization overheads, negating the benefits of compression and further increasing the computational or storage burden.

To overcome this, we aim to design an acceleration scheme that uses mixed precision without requiring coding or decoding. As shown in Figure 3, a large proportion of DNN parameters can be accurately represented with 4 bits or fewer. By dividing quantized values into 4-bit segments, we can split them into low-precision and high-precision parts. Even in activation values that are less favorable for quantization, we

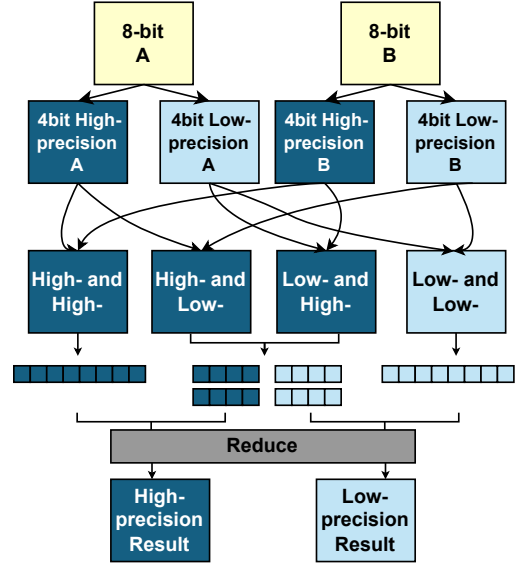


Fig. 5: An example of BLOOM calculation.

observe sparsity of 60% to 80% in the high-precision parts. This opens up new opportunities to explore more efficient quantization and compression strategies. Additionally, this 4-bit segmentation naturally aligns memory and hardware, eliminating the need for complex alignment schemes.

Building on this insight, we propose treating quantized values as independent high- and low-precision parts processed in parallel through bit-slicing, eliminating serialization or deserialization overhead. We introduce BLOOM, a precision-aware bit-slicing framework that splits 8-bit quantized parameters into two 4-bit segments for parallel computation. BLOOM then efficiently merges the results by assigning the appropriate bits to the final output, as illustrated in Figure 5. This interleaved computation leverages the sparsity of high-precision parts for energy savings and computational acceleration, without introducing additional storage or computational overhead.

IV. BLOOM ARCHITECTURE DESIGN

To fully leverage BLOOM's potential, the accelerator architecture must be customized for the framework without relying on encoders or decoders. BLOOM partitions model parameters to exploit inherent sparsity, maximizing acceleration and energy efficiency. Its precision-aware interleaved workflow optimizes computation without additional hardware and seamlessly integrates with mainstream acceleration platforms like systolic arrays. This section details the architecture.

A. Architecture Overview

This section presents how BLOOM integrates in weight-stationary systolic array architecture with minimal modifications. On such an architecture, our proposed BLOOM architecture can directly support the mixed precision, which is efficient for quantizing DNN tensors that have different importance and distribution. The architecture overview is shown in Figure 6, which includes 4-bit PEs in weight-stationary systolic array

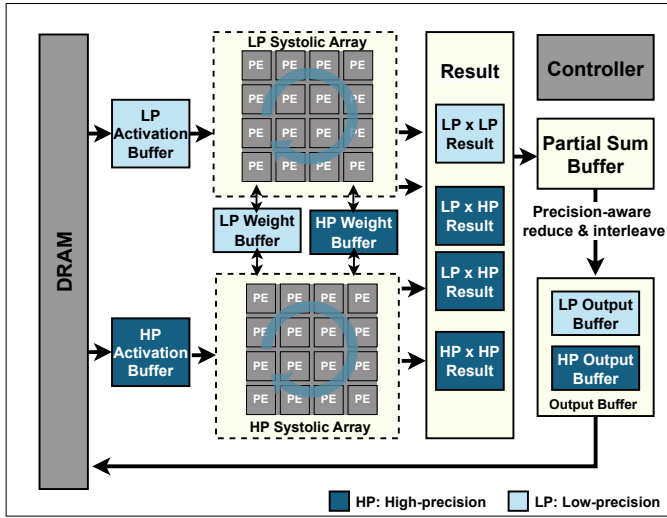


Fig. 6: Overview of BLOOM architecture.

pages, along with input/weight/output/partial sum buffers, an accumulation unit, a controller, and peripheral circuits.

A key feature of BLOOM is its precision-aware bit-slicing of model parameters into two PE groups, which are responsible for computing the high-precision (HP) and low-precision (LP) parts, respectively. These parts are interleaved during computation and stored in their respective buffers but remain separate until they converge to finalize the computation. This design allows BLOOM to fully leverage the inherent sparsity within the high-precision parts, thereby maximizing performance.

B. Joint support for the mixed-precision computation

BLOOM uses the bit-slice mechanism to separate data into a sparse high-precision part and a dense low-precision part. These parts are processed in parallel on distinct PE arrays, where the corresponding weights are also preloaded. This setup allows all computations to be performed in 4-bit precision, minimizing both computation and storage overhead. Based on the nature of the data, there are three types of computation: high-precision with high-precision, low-precision with low-precision, and high-precision with low-precision.

- **High-precision with high-precision** computations: BLOOM leverages the sparsity in the high-precision part to remove unnecessary block-wise calculations.
- **Low-precision with low-precision** computations: BLOOM performs dense 4-bit matrix multiplications, which significantly reduces computational and memory costs compared to 8-bit operations that incur additional overhead due to memory alignment requirements.
- **Low-precision with high-precision** computations: BLOOM can still benefit from the sparsity in the high-precision data. However, to maintain a balance between the two types of operations, some computations may be delegated to the high-precision array. This is because the larger sparsity in high-precision data can result in lower workloads. Such a flexible scheme ensures efficient workload balancing while still utilizing sparse acceleration where applicable. By default, BLOOM

assigns high-precision computations to the HP array and low-precision computations to the LP array.

Through such an adaptive and precision-aware approach, BLOOM maximizes sparsity benefits while maintaining flexibility and efficiency across varying data precision requirements.

C. Execution flow of BLOOM

Figure 6 also illustrates the execution flow of BLOOM, which involves the following five steps:

- **Data Slicing:** The inputs are divided into 4-bit high-precision and 4-bit low-precision parts. Such a partitioning is performed in storage using the corresponding buffers, without any encoding processes.
- **Weight Setup:** Weights are pre-split into high-precision and low-precision components and stored in the respective buffers. These weights are fixed in the systolic array prior to computation.
- **Parallel Computation:** The partitioned inputs and weights are processed in a 4-bit PE array. The high-precision data, due to its inherent sparsity, benefits from acceleration mechanisms, while the low-precision data is handled in a dense format. As detailed in Section IV-B, computations are categorized into three types, with the majority benefiting from the sparsity in high-precision data.
- **Partial Sum Accumulation:** Instead of directly aggregating results from each case, BLOOM accumulates partial sums in a buffer. This approach ensures intermediate results are managed efficiently.
- **Precision-Aware Reduction:** BLOOM performs a reduction operation on the accumulated partial sums to produce 8-bit output results. These results are then interleaved, with the high-precision and low-precision parts separated and sent to the output buffer.

Overall, BLOOM efficiently handles models with outliers without wasting number representations. By leveraging the inherent sparsity of the high-precision parts and using interleaving techniques in computation and storage, BLOOM avoids additional costs. This design allows a memory-aligned architecture to manage outlier values with minimal hardware overhead while achieving significant performance gains. BLOOM can be seamlessly integrated into existing hardware accelerators, such as systolic arrays.

D. Workload-aware PE Configuration

Due to the varying workloads between high-precision and low-precision computations, the completion times of the two types of PE arrays in BLOOM differ. This makes proportional workload allocation critical. Although both types use 4-bit PEs with the same computational speed, the sparsity in the high-precision data results in a lower workload. Therefore, more PE arrays must be allocated to low-precision computations, or additional tasks should be assigned to high-precision PE arrays to prevent bottlenecks.

However, dynamically adjusting the workload during runtime may introduce additional overhead, potentially offsetting

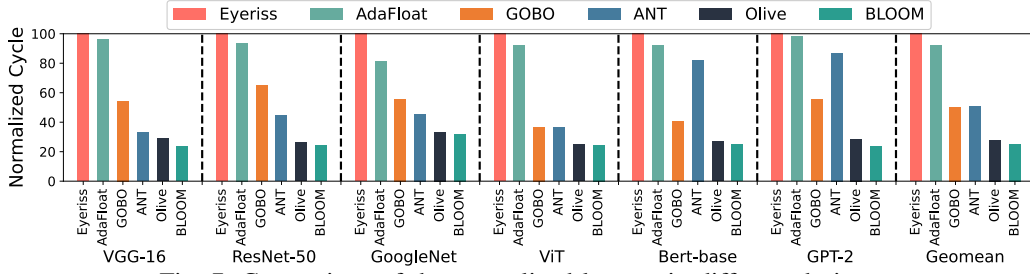


Fig. 7: Comparison of the normalized latency in different designs.

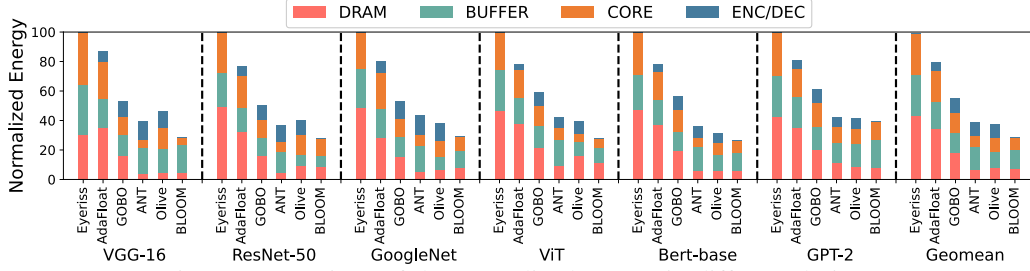


Fig. 8: Comparison of the normalized energy in different designs.

the performance gains. To end this, we analyze the workloads of high-precision and low-precision parts within weights of offline. By examining the workload distribution, we can allocate the appropriate number of PEs for high- and low-precision tasks for different models, ensuring that the processing times for both PE arrays are as balanced as possible.

For mixed-precision computations, this offline analysis allows for flexible task deployment, which helps optimize overall efficiency and maintain a balance between different types of computations. This approach ensures that performance is maximized across various workloads. Further details on this balance will be explored in Section V-C.

V. EVALUATION

A. Methodology

Benchmark. To evaluate the performance of the BLOOM framework, we implement it using the PyTorch [23] framework. We select three representative networks: VGG-16, ResNet-50, and GoogleNet, running them on the ImageNet dataset [24]. To demonstrate BLOOM’s scalability, we also evaluate mainstream attention-based models: ViT [25] with ImageNet, Bert-base with the GLUE dataset, and GPT2-medium with the WikiText-103 dataset [26]. Additionally, we implement the proposed BLOOM architecture in Verilog and synthesize it using Synopsys Design Compiler [27] to estimate chip area and total power consumption under 45nm technology at 500MHz [28]. We also use CACTI [29] to model the energy and area of each on-chip buffer.

Baseline. We compare BLOOM with the following five baselines: 1) Eyeriss [30], [31], which is a spatial energy-efficient dataflow architecture. It uses coarse-grained quantization of INT16 throughout the network, and it is a mainstream NN accelerator architecture. In the evaluation, we will use it as the baseline of the standard DNN accuracy. 2) AdaptiveFloat [32], or AdaFloat, which dynamically clips parameter range at a layer granularity to create faithful encodings, and requires

TABLE I: Detailed result of evaluated model and dataset.

Type	CNN-based			Attention-based		
Model	VGG-16	ResNet-50	GoogleNet	ViT	BERT	GPT2-XL
Dataset	ImageNet			SST-2	Wikitext	
FP-32 Acc.(%)	71.59	76.15	62.46	84.19	93.23	16.94 (PPL)
BLOOM Acc.(%)	71.59	76.05	62.35	84.03	92.78	17.15 (PPL)

an 8-bit float to maintain the original model accuracy; 3) GOBO [10], which employs a list of coordinates to indicate the locations of different data representations, using high-precision quantization for larger data representations. 4) ANT [33], which is a hardware-friendly quantization accelerator that combines the power-of-two data types and INT type for low bit-width; 5) Olive [11], which is outlier-aware quantization accelerator. We take reported results in their paper for comparison, and for models that are not available in their paper, we reproduce their experiments and report the results.

B. Experimental Results

Accuracy. For quantized DNN models, the BLOOM framework leverages intrinsic sparsity with minimal accuracy loss. As shown in Table I, BLOOM achieves nearly original accuracy for commonly used vision and language models. On the ImageNet dataset, BLOOM exhibits an average accuracy loss less than 0.1% against the original FP32 model. And for LLM like GPT2, deploying the BLOOM framework also has nearly no effect on the perplexity. This minimal impact on accuracy is due to BLOOM’s use of an 8-bit format for computations, which does not alter the computation process or results. The experimental results demonstrate that BLOOM effectively maintains model accuracy while delivering performance acceleration, even for LLM.

Performance. Figure 7 illustrates the normalized total execution cycles across different accelerators for six networks. BLOOM shows the most significant performance improvement. Compared to other accelerators, BLOOM achieves

TABLE II: Configurations of different accelerators under 45nm process.

	Eyeriss [30]	GOBO [10]	ANT [33]	Olive [11]	BLOOM
#PEs	224	4096	4096	4096	4096
Bitwidth	INT16	INT4	INT4	INT4	INT4
Decoder	-	256	128	192	-
Area(mm ²)	0.32	0.417	0.327	0.338	0.32

substantial speedups, with average improvements of $3.96\times$, $3.64\times$, $1.99\times$, $2.01\times$, and $1.20\times$ over Eyeriss, AdaFloat, GOBO, ANT, and Olive, respectively. Notably, BLOOM achieves a $3.69\times$ speedup over ANT on GPT2, highlighting its efficiency. GOBO and ANT perform less effectively on most models, with ANT especially struggling with attention-based models due to their complex coding and decoding mechanisms. Olive, despite its optimization for decoding, suffers from a complex encoding process and a limited percentage of outlier pairs, resulting in less impressive speedups compared to BLOOM. This is because BLOOM eliminates the serialization and deserialization overhead typically associated with encoding and decoding, which leads to a significant acceleration of calculations.

Energy. Figure 8 shows the normalized energy consumption across six networks, broken down into DRAM, global buffer (labeled as BUFFER), processing core (labeled as CORE), and encoder/decoder (labeled as ENC/DEC). BLOOM exhibits the lowest energy consumption for both CNN-based and attention-based models. On average, BLOOM consumes 71.3%, 64.4%, 48.4%, 26.8%, and 24.6% less energy compared to Eyeriss, AdaFloat, GOBO, ANT, and Olive, respectively. Since other accelerators require complex encoder/decoder mechanisms that add significant energy overhead, BLOOM’s absence of these components provides a clear advantage. Even with attention-based models, which have a larger number of parameters, BLOOM maintains substantial energy efficiency due to its simple and efficient interleaved computation flow.

Area. Table II provides a comparison of the area and number of PEs for BLOOM and other accelerators, all scaled to a 45-nm process. The table also considers the area required for decoders, where applicable. BLOOM maintains a consistent total PE area with SA while minimizing area overhead. Other accelerators, such as GOBO, incur significant area overhead due to the inclusion of decoders. In contrast, BLOOM’s use of compact 4-bit PEs and its lack of a decoder result in the lowest area overhead. This design choice allows BLOOM to fit more PEs within the same area budget, leading to enhanced computational performance.

C. Design Space Exploration

This section discusses workload balancing between high- and low-precision PE arrays, as introduced in Section IV-B and Section IV-D. We then analyze BLOOM’s scalability by evaluating its performance across various model sizes.

Workload-aware PEs configuration. To evaluate the performance gains of BLOOM under different scenarios and determine the optimal ratio of high- to low-precision PE arrays, we conducted experiments with three models. We compared

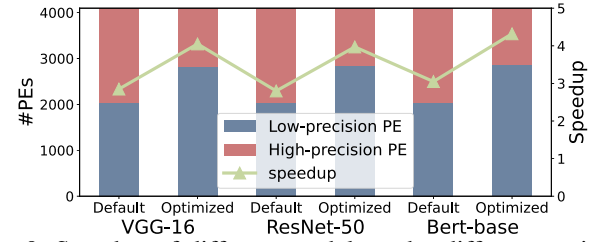


Fig. 9: Speedup of different models under different settings.

two cases: 1) assigning an equal number of PE arrays for high- and low-precision computations, and 2) adjusting the number of PE arrays for each precision type to ensure that the high-precision and low-precision PE arrays can complete their computations nearly simultaneously.

Figure 9 illustrates the results. Allocating an equal number of high- and low-precision PE arrays yields a significant speedup of approximately $3\times$, primarily due to BLOOM’s use of 4-bit PEs. However, due to the inherent computational speed differences between the two types of arrays, low-precision computations can become a bottleneck. By adjusting the PE array allocation, we enhance the computational parallelism of the low-precision part, achieving a more pronounced speedup of over $4\times$. The effective allocation of PE arrays leverages BLOOM’s advantages, resulting in significant improvements in computational efficiency.

Scalability of BLOOM. We analyze how increasing model sizes affect the computation speedup achieved by BLOOM. As shown in Figure 10, as model sizes increase, the computation speedup with BLOOM framework also rises. Such a trend is attributed to BLOOM’s architecture design, where higher sparsity can get a greater pronounced speedup effect. Notably, as the model size increases, the sparsity naturally embedded in the parameters increases, which further emphasizes the advantages of the BLOOM architecture. These findings demonstrate BLOOM’s scalability and consistent performance in enhancing energy efficiency, especially for transformer-based models with large parameter sizes.

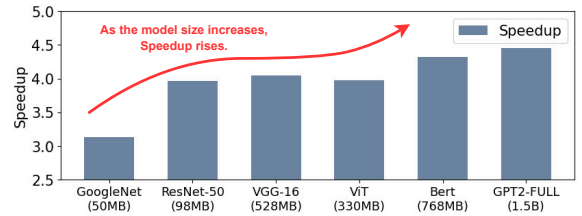


Fig. 10: Speedup with different model size.

VI. CONCLUSION

We introduce BLOOM, a precision-aware bit-slicing framework for efficient DNN acceleration with minimal hardware overhead. BLOOM leverages the inherent sparsity in high-precision quantized values caused by outliers, avoiding serialization and deserialization costs. It seamlessly integrates with existing platforms through interleaved computational streams, enabling mixed-precision encoding without information loss. Evaluations show that BLOOM outperforms similar quantization schemes in performance, energy efficiency, and accuracy.

REFERENCES

- [1] G. Montavon, W. Samek, and K.-R. Müller, “Methods for interpreting and understanding deep neural networks,” *Digital signal processing*, vol. 73, pp. 1–15, 2018.
- [2] E. Ozen and A. Orailoglu, “Evolving complementary sparsity patterns for hardware-friendly inference of sparse dnns,” in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–8.
- [3] F. Liu, W. Zhao, Y. Chen, Z. Wang, Z. He, R. Yang, Q. Tang, T. Yang, C. Zhuo, and L. Jiang, “Pim-dh: Reram-based processing-in-memory architecture for deep hashing acceleration,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 1087–1092.
- [4] L. YUAN, C. HUI, Y. WU, R. LIAO, F. JIANG, and Y. GAO, “Video enhancement network based on cnn and transformer,” *ZTE Communications*, vol. 22, no. 4, p. 78, 2024.
- [5] L. Deng and Y. Zhao, “Deep learning-based semantic feature extraction: A literature review and future directions,” *ZTE communications*, vol. 21, no. 2, p. 11, 2023.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [8] OpenAI, “Gpt-4 technical report,” 2023.
- [9] F. Liu *et al.*, “Improving neural network efficiency via post-training quantization with adaptive floating-point,” in *ICCV*, 2021.
- [10] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
- [11] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [12] Z. Song, B. Fu, F. Wu, Z. Jiang, L. Jiang, N. Jing, and X. Liang, “Drq: dynamic region-based quantization for deep neural network acceleration,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 1010–1021.
- [13] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, “Hq: Hardware-aware automated quantization with mixed precision,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 8612–8620.
- [14] S.-E. Chang, Y. Li, M. Sun, R. Shi, H. K.-H. So, X. Qian, Y. Wang, and X. Lin, “Mix and match: A novel fpga-centric deep neural network quantization framework,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 208–220.
- [15] F. Liu, W. Zhao, Z. Wang, Y. Chen, Z. He, N. Jing, X. Liang, and L. Jiang, “Ebsp: evolving bit sparsity patterns for hardware-friendly inference of quantized deep neural networks,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 259–264.
- [16] E. Park, D. Kim, and S. Yoo, “Energy-efficient neural network accelerator based on outlier-aware low-precision computation,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 688–698.
- [17] S. Jain, S. Venkataramani, V. Srinivasan, J. Choi, K. Gopalakrishnan, and L. Chang, “Biscaled-dnn: Quantizing long-tailed datastructures with two scale factors for deep neural networks,” in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.
- [18] F. Liu, N. Yang, H. Li, Z. Wang, Z. Song, S. Pei, and L. Jiang, “Spark: Scalable and precision-aware acceleration of neural networks via efficient encoding,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 1029–1042.
- [19] J. Shen, K. Jiang, and X. Tan, “Boundary data augmentation for offline reinforcement learning,” *ZTE Communications*, vol. 21, no. 3, p. 29, 2023.
- [20] N. Yang, F. Liu, Z. Wang, J. Zhao, and L. Jiang, “Searchq: Search-based fine-grained quantization for data-free model compression,” *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, 2024.
- [21] X. Wei, Y. Zhang, X. Zhang, R. Gong, S. Zhang, Q. Zhang, F. Yu, and X. Liu, “Outlier suppression: Pushing the limit of low-bit transformer language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 402–17 414, 2022.
- [22] F. Liu, N. Yang, Z. Song, Z. Wang, H. Li, S. Huang, Z. Song, S. Pei, and L. Jiang, “Inspire: Accelerating deep neural networks via hardware-friendly index-pair encoding,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [24] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [25] A. Dosovitskiy, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [26] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [27] S. D. Compiler, [Online]. Available: <https://www.synopsys.com/support/training/rtlsynthesis/design-compiler-rtl-synthesis.html>, 2019.
- [28] D. Fujiki, S. Mahlke, and R. Das, “In-memory data parallel processor,” *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 1–14, 2018.
- [29] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “Cacti 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [30] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.
- [31] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.
- [32] T. Tambe, E.-Y. Yang, Z. Wan, Y. Deng, V. J. Reddi, A. Rush, D. Brooks, and G.-Y. Wei, “Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [33] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.